

Measuring performance

João Luís Ferreira Sobral
www.di.uminho.pt/~jls
jls@di.uminho.pt

Oct/2023

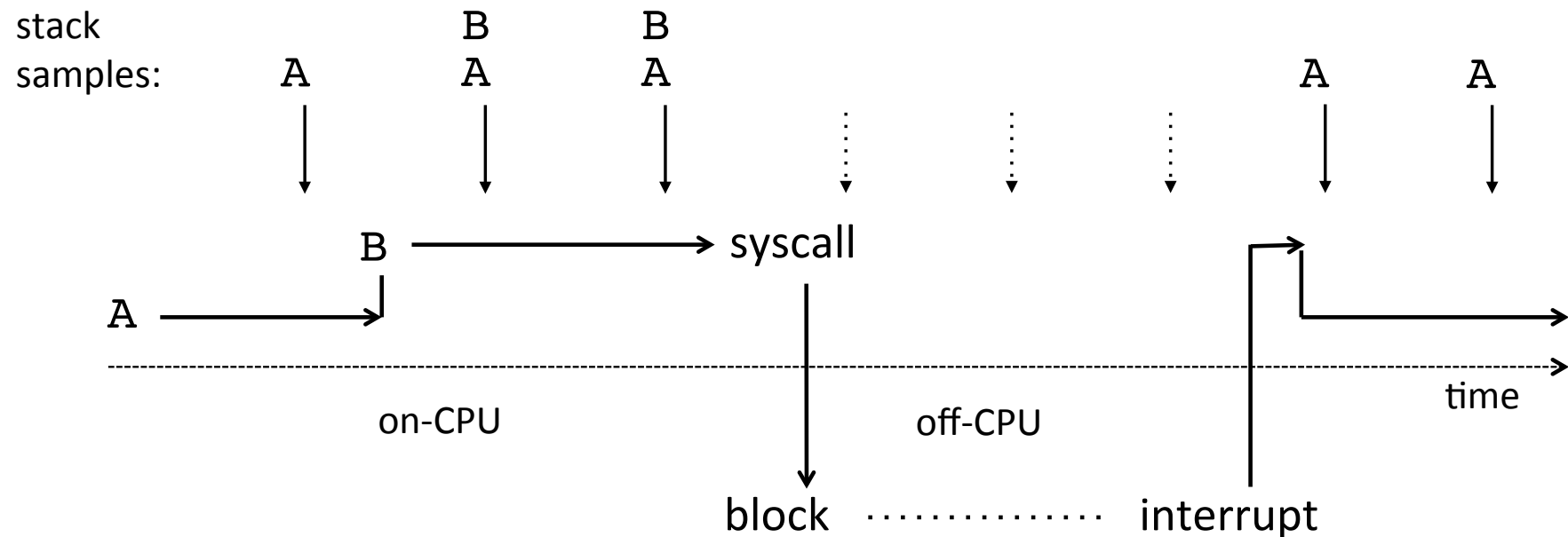
Performance of applications

Experimental study

- **Sequential execution profile:**
 - Identify application **hot-spots**
 - Functions that take most of the time to execute
 - Can be implemented by specific tools or by directly instrumenting the code
 - There is always an overhead introduced in the base application

CPU profiling using sampling

- Record stacks at a timed interval: simple and effective
 - Pros: Low (deterministic) overhead
 - Cons: Coarse accuracy, but usually sufficient



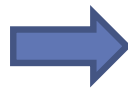
Profiling code execution

Techniques to measure the application time-profile (profiling)

● Polling (sampling)

- the application is periodically interrupted to collect performance data
- Example: gprof

Flat view



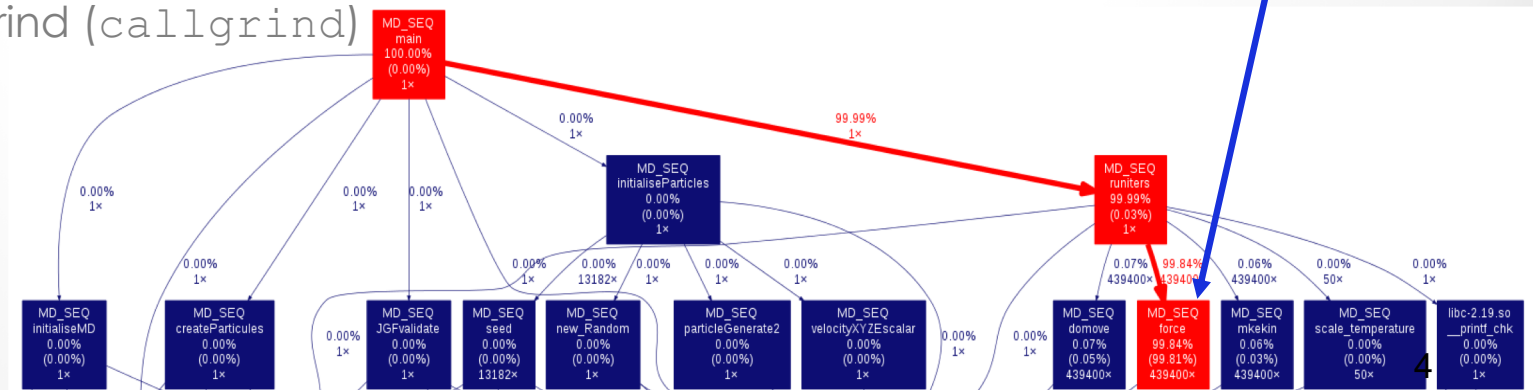
% time	cumulative seconds	self seconds	calls	self s/call	total s/call	name
50.00	2.47	2.47	2	1.24	1.24	matSort
24.70	3.69	1.22	1	1.22	1.22	matCube
24.70	4.91	1.22	1	1.22	1.22	sysCube
0.61	4.94	0.03	1	0.03	4.94	main
0.00	4.94	0.00	2	0.00	0.00	vecSort
0.00	4.94	0.00	1	0.00	1.24	sysSort
0.00	4.94	0.00	1	0.00	0.00	vecCube

Hot-spot

● Instrumentation

- programmer (or tools) introduce code to collect performance data on useful events
 - may produce better results but also produces more interference (e.g., overhead)
- Example: Valgrind (callgrind)

Tree view

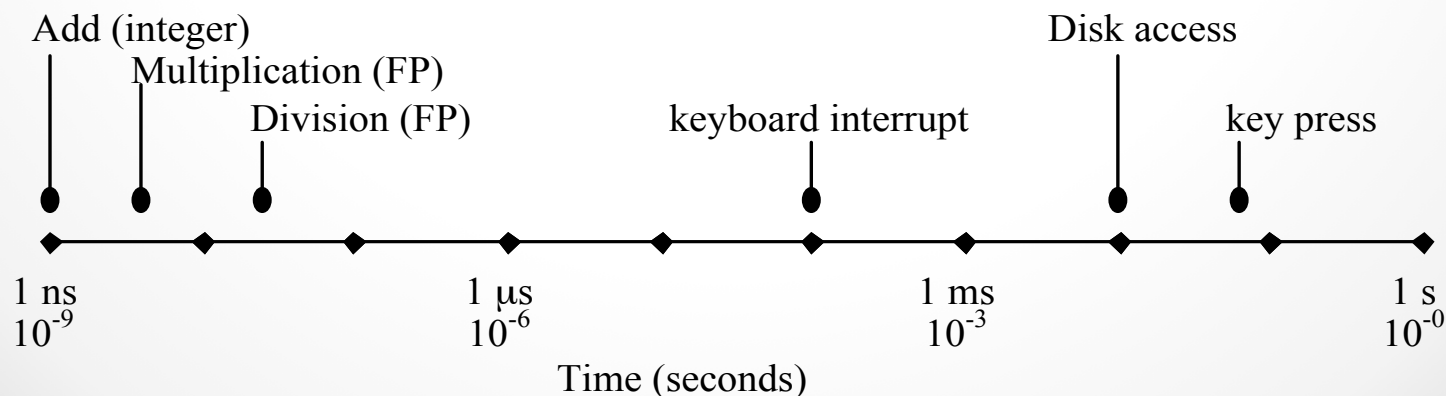


Measuring performance

• Principles

- **Isolate from external factors**
 - Consider the measurement overhead
 - Repeat the measurement
 - Avoid other system load
- **Document the experiment to be reproducible by others**
 - Hardware, software versions, system state...
- **Important: clock resolution**
 - Precision: difference between measured and real time
 - Resolution: time unit between clock increments
 - In principle, not possible to measure events shorter than the clock resolution, but...

• Event timescale (1GHz machine)



Measuring performance

- **How much time is required to execute an application?**

- **CPU time**

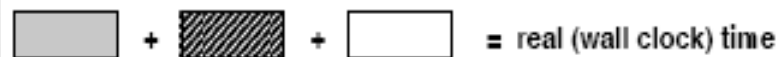
- Time dedicated exclusively to program execution
- Does not depend on other activities in the system

- **Wall time**

- Time measured since the start until the end of execution
- Depends on the system load, I/O, ...

- **Complexities**

- Process scheduling (10ms?)
- Load introduced by other processes (e.g., garbage collector in JVM, other users)



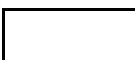
We will use the word "time" to refer to user time.



real (wall clock) time

 = user time (time executing instructions in the user process)

 = system time (time executing instructions in kernel on behalf of user process)

 = some other user's time (time executing instructions in different user's process)

Measuring performance

- Options to measure execution time

- “time” command line

- Only for measurements >1 seg

```
[x@search7edu]$ time ./a.out
real    0m0.938s  <= wall time
user    0m0.934s
sys     0m0.001s
```

- Time given by “perf stat”

```
srunk --partition=cpar perf stat -M cpi ./a.out
```

```
141556121955      inst_retired.any      #      0,9 CPI
122749194495      cycles
```

```
38,355552647 seconds time elapsed  <= Wall time
```

```
38,335463000 seconds user
0,002000000 seconds sys
```

- Timer function in OpenMP / MPI

- omp_get_wtime, omp_get_wtick*
- MPI_Wtime*

- System.nanoTime()** (in Java 4)

High resolution
implementations!

Measuring performance

- **How to combine results from several measurements?**

- **Average**

- Affected by extreme high/low values
- Additionally: show the deviation among measurements (*standard deviation*)
- Example:

```
srun --partition=cpar perf stat -r 5 -M cpi ./a.out
```

```
...
          410257709      inst_retired.any      # 0,9 CPI      ( +- 0,01% )
          354731534      cycles                  ( +- 0,20% )

          0,110387 +- 0,000399 seconds time elapsed ( +- 0,36% )
```

- **Best measure**

- Value in ideal conditions

- **Average of k-best**

- k-best within a given tolerance
- Best guarantee for reproducible results
- Removes outliers

- **Median**

- More robust to large variations